GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: SGC-CMG 4RF

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	SGC-CMG
Equipo de Trabajo	InnovaCode G5
Integrantes	Brayan Gualonuña David Rodriguez Steven Allauca
Propósito	Guiar al estudiante para pasar del análisis funcional al análisis estructural y dinámico, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso. Su finalidad es mostrar cómo un CRUD de Estudiante puede analizarse de forma progresiva mediante tres modelos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis no está en programar la interfaz, sino en comprender qué responsabilidad cumple cada elemento del modelo y cómo se conserva la trazabilidad desde el requisito funcional hasta la interacción entre objetos.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación del CRUD del estudiante	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: SGC-CMG

Dato	Tipo sugerido	Descripción	Validación mínima
Campeonato	Texto corto o entero	Torneo deportivo oficial donde se configuran categorías (Senior, Máster, Femenino), cupos máximos, rango de fechas del certamen y costos oficiales de inscripción y arbitraje.	Nombre oficial único en la temporada, fechas cronológicas válidas (fecha_inicio <= fecha_fin) y valores de inscripción ($60) y arbitraje ($15) mayores a cero.

Equipo y Jugador	Registro de inscripción	Club participante y nómina acreditada de hasta 18 deportistas con indumentaria oficial, documentos de identidad (cédula y foto) y carnet QR inteligente.	Nombre de equipo único en la categoría, cédula ecuatoriana válida (módulo 10), mayoría de edad reglamentaria (>= 18 o >= 40 años), archivos PDF/foto <= 2 MB y carnet QR generado.
Jornada y Fixture	Programación deportiva	Calendario oficial de fechas y emparejamientos deportivos generados mediante algoritmo Round Robin, con asignación de canchas, horarios y ternas arbitrales.	Equipos habilitados sin repetición en la misma jornada, asignación de cancha disponible sin solapamiento de horarios y árbitro asignado sin cruce de turnos.
Tabla de Posiciones	Consolidado estadístico	Registro acumulado de la clasificación deportiva computado automáticamente a partir de las actas de juego digitales cerradas.	Ecuaciones de consistencia matemática cumplidas (PJ = PG + PE + PP, DG = GF - GC, PTS = PG*3 + PE*1) y autorización administrativa previa para publicación web.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Configurar Torneo	CU-01: Administrar campeonato	CU-01: Administrar campeonato
Inscribir Equipo y Nómina	CU-02: Administrar equipos y jugadores	Permite al Capitán cargar la nómina de hasta 18 jugadores con cédulas y fotos, facilitando al Administrador la auditoría y emisión del carnet digital QR.
Generar Fixture	CU-03: Administrar jornadas y fixture	Permite al Administrador disparar el algoritmo Round Robin para programar los encuentros, asignando canchas y árbitros sin colisión de horarios.
Publicar Clasificación	CU-04: Administrar tabla de posiciones	Permite auditar los marcadores tras el cierre de actas, recalcular puntajes y gol diferencia de forma automática, y autorizar la publicación oficial para consulta pública.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir administrar campeonatos deportivos, incluyendo creación, configuración de categorías (Senior, Máster, Femenino), definición de cupos de equipos, establecimiento del rango de fechas del torneo y registro de costos de inscripción ($60) y arbitraje semanal ($15).	Administrador académico	Administrar Campeonato
RF-02	El sistema debe permitir administrar equipos y jugadores, registrando clubes deportivos, nóminas de hasta 18 futbolistas con indumentaria oficial, validando documentación de identidad (cédula PDF y foto JPG <= 2 MB) y emitiendo carnets digitales con código QR inteligente.	Capitán de Equipo	Administrar Equipos y Jugadores
RF-03	El sistema debe permitir administrar jornadas y fixtures, generando automáticamente el calendario de partidos mediante algoritmo Round Robin para hasta 32 equipos, asignando canchas sin solapamiento horario y designando ternas arbitrales sin cruce de turnos.	Administrador académico	Administrar Jornadas y Fixture
RF-04	El sistema debe permitir administrar la tabla de posiciones, calculando automáticamente puntajes, gol diferencia y juego limpio tras el cierre de actas digitales, auditando resultados y publicando clasificaciones oficiales en el portal web para aficionados y jugadores.	Administrador del Complejo	Administrar Tabla de Posiciones

RF-05	El sistema debe validar los datos obligatorios antes de agregar o actualizar.	Sistema	Validar datos del estudiante

5. Diagrama de casos de uso de análisis

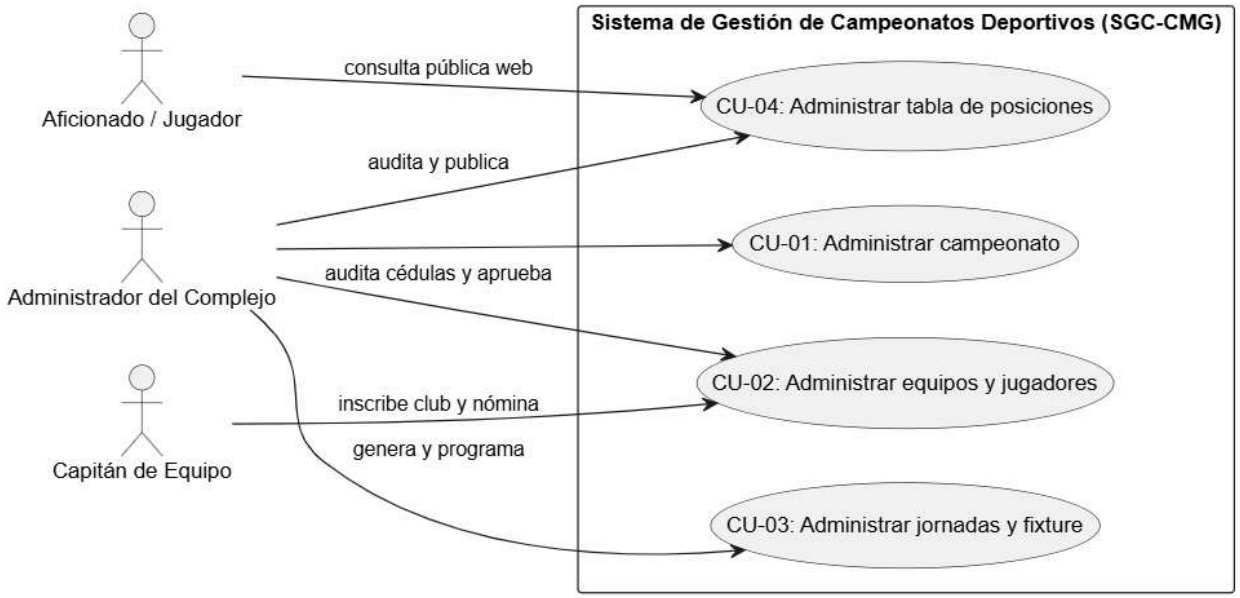
Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador del Complejo" as Admin
actor "Capitán de Equipo" as Capitan
actor "Aficionado / Jugador" as Aficionado

rectangle "Sistema de Gestión de Campeonatos Deportivos (SGC-CMG)" {
  usecase "CU-01: Administrar campeonato" as UC1
  usecase "CU-02: Administrar equipos y jugadores" as UC2
  usecase "CU-03: Administrar jornadas y fixture" as UC3
  usecase "CU-04: Administrar tabla de posiciones" as UC4
}

Admin --> UC1
Capitan --> UC2 : inscribe club y nómina
Admin --> UC2 : audita cédulas y aprueba
Admin --> UC3 : genera y programa
Admin --> UC4 : audita y publica
Aficionado --> UC4 : consulta pública web
@enduml
```



6. Derivación de clases de análisis

A partir de los casos de uso se identifican clases de análisis. Para mantener una separación clara de responsabilidades se recomienda usar los estereotipos boundary, control y entity. Además, se puede incluir un repositorio para representar la persistencia o colección de estudiantes sin entrar aún en detalles técnicos de base de datos.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del actor, valida entradas en cliente y muestra resultados.	FormularioCampeonato FormularioEquipo FormularioFixture FormularioTablaPosiciones	Representa los formularios web para configurar torneos, registrar nóminas con fotos/PDF, visualizar el calendario y consultar las posiciones.
Control	Coordina la lógica de negocio, orquesta las reglas y comanda la persistencia.	ControlCampeonato ControlEquipo ControlFixture ControlTablaPosicione	Aplica validaciones de módulo 10, fechas cronológicas, algoritmo combinatorio Round Robin y ecuaciones estadísticas (PTS = PG*3 + PE*1).
Entity / Entidad	Modela la información y conceptos centrales persistentes del dominio.	Campeonato Equipo (y Jugador) Fixture TablaPosiciones	Encapsula los estados del torneo, nóminas acreditadas, carnets QR emitidos, partidos programados y clasificaciones deportivas acumuladas.
Repositorio / Colección	Gestiona el almacenamiento, recuperación y persistencia de las entidades.	RepositorioCampeonato RepositorioEquipo RepositorioFixture RepositorioTablaPosiciones	Provee servicios de consulta y guardado sin acoplar el análisis a sentencias SQL ni esquemas físicos de base de datos.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

' Frontera (Boundary)
class "FormularioCampeonato" <<boundary>> {
  - txtNombre: String
  - cmbCategorias: List<String>
  - txtCupos: int
  - dtpFechaInicio: Date
  - dtpFechaFin: Date
  - txtCostoInscripcion: double
  - txtCostoArbitraje: double
  + clickRegistrarTorneo()
  + mostrarConfirmacion(mensaje: String)
}

class "FormularioEquipo" <<boundary>> {
  - txtNombreClub: String
  - txtColorTitular: String
  - txtColorAlterno: String
  - gridJugadores: List<Object>
  - fileCedulaPdf: File
```

```
  - fileFotoJpg: File
  + clickInscribirEquipo()
  + clickValidarDocumentos()
  + mostrarEstadoCarnets(qrBase64: String)
}

class "FormularioFixture" <<boundary>> {
  - cmbCategoria: String
  - gridPartidos: List<Object>
  - cmbCanchas: List<String>
  - cmbArbitros: List<String>
  + clickGenerarRoundRobin()
  + clickPublicarCalendario()
}

class "FormularioTablaPosiciones" <<boundary>> {
  - cmbCategoria: String
  - tblClasificacion: List<Object>
  + clickAuditarActas()
  + clickPublicarWeb()
  + mostrarTablaPublica()
}

' Control
class "ControlCampeonato" <<control>> {
  + registrarCampeonato(nombre: String, cat: List<String>, cupos: int, fIni: Date, fFin: Date, cInsc:
double, cArb: double): boolean
  + validarRangoFechas(fIni: Date, fFin: Date): boolean
}

class "ControlEquipo" <<control>> {
  + registrarClub(nombre: String, cTit: String, cAlt: String): boolean
  + agregarJugador(cedula: String, nombres: String, fNac: Date, pdf: File, foto: File): boolean
  + validarCedulaModulo10(cedula: String): boolean
  + emitirCarnetQR(idJugador: String): String
}

class "ControlFixture" <<control>> {
  + generarFixtureRoundRobin(idCat: String): List<Object>
  + asignarCanchasHorarios(idPart: String, idCan: String, hora: Time): boolean
  + verificarDisponibilidadArbitro(idArb: String, hora: Time): boolean
}

class "ControlTablaPosiciones" <<control>> {
  + recalcularEstadisticas(idCat: String): boolean
  + procesarActasCerradas(): void
  + autorizarPublicacion(idCat: String): boolean
}
```

```
' Entidad
class "Campeonato" <<entity>> {
  - idCampeonato: String
  - nombre: String
  - estado: String
  - fechaInicio: Date
  - fechaFin: Date
  - costoInscripcion: double
  - costoArbitraje: double
}

class "Equipo" <<entity>> {
  - idEquipo: String
  - nombreClub: String
  - colorTitular: String
  - colorAlterno: String
  - habilitado: boolean
}

class "Jugador" <<entity>> {
  - cedula: String
  - nombres: String
  - fechaNacimiento: Date
  - carnetQrHash: String
}

class "Fixture" <<entity>> {
  - idFixture: String
  - jornadaNum: int
  - fechaHora: DateTime
  - canchaAsignada: String
  - arbitroDesignado: String
  - estadoPartido: String
}

class "TablaPosiciones" <<entity>> {
  - idTabla: String
  - pj: int
  - pg: int
  - pe: int
  - pp: int
  - gf: int
  - gc: int
  - dg: int
  - puntos: int
  + calcularPuntos(): int
}
```

```
' Repositorio
class "RepositorioCampeonato" <<repository>> {
  + guardar(camp: Campeonato): void
  + existeNombre(nombre: String): boolean
  + obtenerActivo(): Campeonato
}

class "RepositorioEquipo" <<repository>> {
  + guardarEquipo(eq: Equipo): void
  + guardarJugador(eqId: String, jug: Jugador): void
  + listarPorCategoria(idCat: String): List<Equipo>
}

class "RepositorioFixture" <<repository>> {
  + guardarCalendario(fix: List<Fixture>): void
  + verificarSolapamiento(cancha: String, hora: DateTime): boolean
}

class "RepositorioTablaPosiciones" <<repository>> {
  + guardarTabla(tabla: TablaPosiciones): void
  + obtenerPorCategoria(idCat: String): TablaPosiciones
}

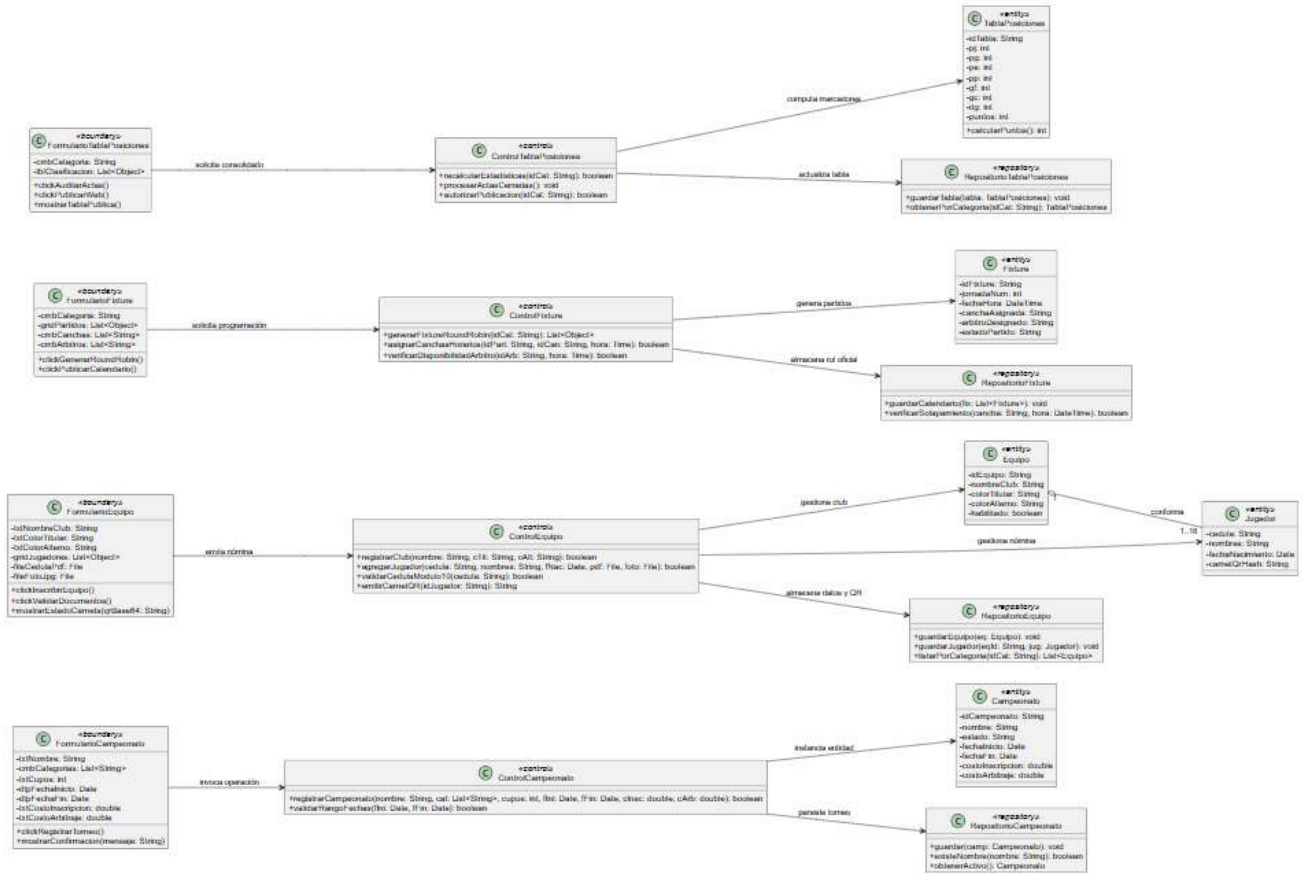
' Relaciones
FormularioCampeonato --> ControlCampeonato : invoca operación
ControlCampeonato --> Campeonato : instancia entidad
ControlCampeonato --> RepositorioCampeonato : persiste torneo

FormularioEquipo --> ControlEquipo : envía nómina
ControlEquipo --> Equipo : gestiona club
ControlEquipo --> Jugador : gestiona nómina
ControlEquipo --> RepositorioEquipo : almacena datos y QR

FormularioFixture --> ControlFixture : solicita programación
ControlFixture --> Fixture : genera partidos
ControlFixture --> RepositorioFixture : almacena rol oficial

FormularioTablaPosiciones --> ControlTablaPosiciones : solicita consolidado
ControlTablaPosiciones --> TablaPosiciones : computa marcadores
ControlTablaPosiciones --> RepositorioTablaPosiciones : actualiza tabla

Equipo "1" o-- "1..18" Jugador : conforma
@enduml
```



7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
CU-01: Administrar campeonato	FormularioCampeonato: Captura nombre, categorías, cupos, fechas y costos.	ControlCampeonato: Valida rango de fechas, costos > 0 y ausencia de duplicidad.	Instancia Campeonato y lo guarda en RepositorioCampeonato.	Secuencia 8.1: Cargar datos -> validar fechas -> verificar unicidad -> instanciar torneo -> guardar -> confirmar estado "Inscripción Abierta".
CU-02: Administrar equipos y CU-03: Administrar jornadas y fixture	FormularioEquipo: Carga club, nómina (hasta 18), fotos y cédulas (PDF <= 2 MB).	ControlEquipo: Valida módulo 10, peso de archivos, cupos y genera firma QR HMAC.	Instancia Equipo y Jugador, almacenándolos en RepositorioEquipo.	Secuencia 8.2: Subir nómina -> validar cédula y formatos -> auditar documento -> generar hash QR -> guardar nómina -> emitir carnet digital.
CU-03: Administrar jornadas y fixture	FormularioFixture: Solicita generación de fecha y visualiza asignaciones.	ControlFixture: Ejecuta algoritmo Round Robin, evita colisiones de canchas y turnos de árbitros.	Crea instancias de Fixture y persiste en RepositorioFixture.	Secuencia 8.3: Disparar Round Robin -> consultar escuadras activas -> programar cruces -> asignar canchas libres -> asignar árbitros -> guardar fixture oficial.

CU-04: Administrar tabla de posiciones	FormularioTablaPosiciones: Presenta panel de auditoría de actas y tabla pública web.	FormularioTablaPosiciones: Presenta panel de auditoría de actas y tabla pública web.	Lee actas cerradas y actualiza la entidad TablaPosiciones en RepositorioTablaPosiciones.	Secuencia 8.4: Cerrar actas -> solicitar cómputo -> procesar marcadores -> actualizar estadísticas -> auditar resultados -> publicar tabla en la web.

8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se recomienda elaborar un diagrama por cada caso de uso principal para evitar diagramas sobrecargados.

8.1 Secuencia: Administrar campeonato (CU-01)

```
@startuml
title Secuencia de análisis - Administrar campeonato (CU-01)
autonumber
actor "Administrador del Complejo" as Admin
boundary "FormularioCampeonato" as UI
control "ControlCampeonato" as Ctrl
entity "Campeonato" as Camp
database "RepositorioCampeonato" as Repo

Admin -> UI : Ingresa nombre del torneo, categorías, cupos, fechas y costos ($60 / $15)
Admin -> UI : Clic en Configurar Torneo
UI -> Ctrl : registrarCampeonato(nombre, cat, cupos, fIni, fFin, cInsc, cArb)
activate Ctrl

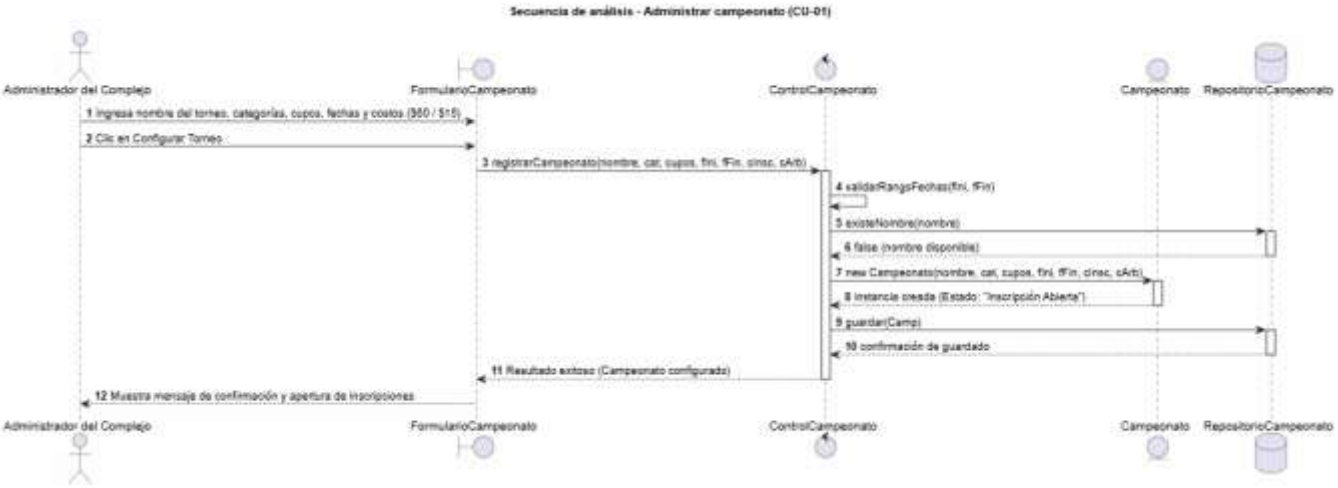
Ctrl -> Ctrl : validarRangoFechas(fIni, fFin)
Ctrl -> Repo : existeNombre(nombre)
activate Repo
Repo --> Ctrl : false (nombre disponible)
deactivate Repo

Ctrl -> Camp : new Campeonato(nombre, cat, cupos, fIni, fFin, cInsc, cArb)
activate Camp
Camp --> Ctrl : instancia creada (Estado: "Inscripción Abierta")
deactivate Camp

Ctrl -> Repo : guardar(Camp)
activate Repo
Repo --> Ctrl : confirmación de guardado
deactivate Repo

Ctrl --> UI : Resultado exitoso (Campeonato configurado)
deactivate Ctrl
```

```
UI --> Admin : Muestra mensaje de confirmación y apertura de inscripciones
@enduml
```



8.2 Secuencia: Administrar equipos y jugadores (CU-02)

```
@startuml
title Secuencia de análisis - Administrar equipos y jugadores (CU-02)
autonumber
actor "Capitán de Equipo" as Capitan
actor "Administrador del Complejo" as Admin
boundary "FormularioEquipo" as UI
control "ControlEquipo" as Ctrl
entity "Equipo" as Eq
entity "Jugador" as Jug
database "RepositorioEquipo" as Repo

Capitan -> UI : Ingresa datos del club e indumentaria
Capitan -> UI : Carga nómina (hasta 18), foto JPG y cédula PDF (<= 2 MB)
Capitan -> UI : Clic en Inscribir Equipo
UI -> Ctrl : registrarClubConNomina(datosClub, listaJugadores)
activate Ctrl

Ctrl -> Ctrl : validarCedulaModulo10(cedula)
Ctrl -> Ctrl : validarPesoArchivos(<= 2 MB)

Admin -> UI : Audita documentación digital
UI -> Ctrl : aprobarDocumentacion(idJugador)

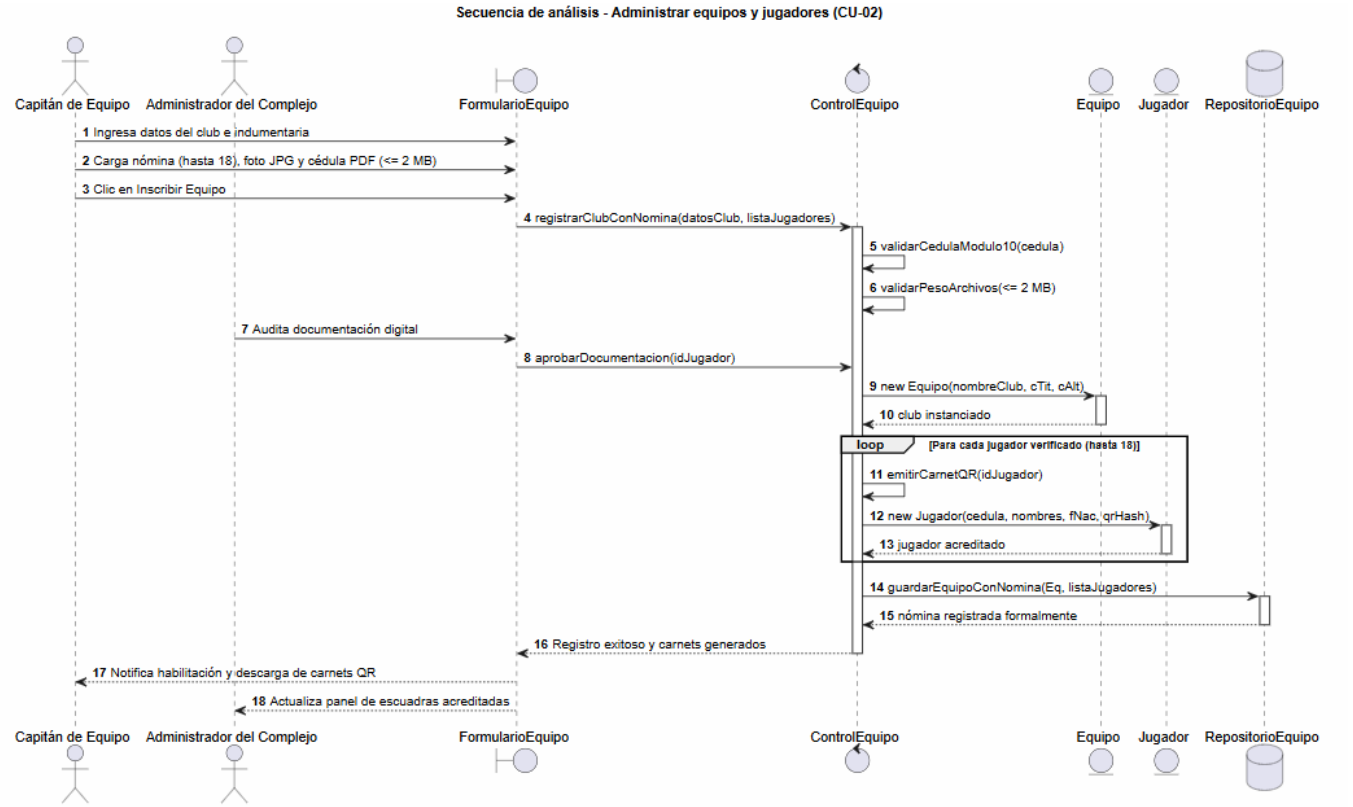
Ctrl -> Eq : new Equipo(nombreClub, cTit, cAlt)
activate Eq
Eq --> Ctrl : club instanciado
deactivate Eq
```

```
loop Para cada jugador verificado (hasta 18)
    Ctrl -> Ctrl : emitirCarnetQR(idJugador)
    Ctrl -> Jug : new Jugador(cedula, nombres, fNac, qrHash)
    activate Jug
    Jug --> Ctrl : jugador acreditado
    deactivate Jug
end

Ctrl -> Repo : guardarEquipoConNomina(Eq, listaJugadores)
activate Repo
Repo --> Ctrl : nómina registrada formalmente
deactivate Repo

Ctrl --> UI : Registro exitoso y carnets generados
deactivate Ctrl

UI --> Capitan : Notifica habilitación y descarga de carnets QR
UI --> Admin : Actualiza panel de escuadras acreditadas
@enduml
```



8.3 Secuencia: Administrar jornadas y fixture (CU-03)

```
@startuml
title Secuencia de análisis - Administrar jornadas y fixture (CU-03)
autonumber
actor "Administrador del Complejo" as Admin
boundary "FormularioFixture" as UI
control "ControlFixture" as Ctrl
entity "Fixture" as Fix
database "RepositorioEquipo" as RepoEq
database "RepositorioFixture" as RepoFix

Admin -> UI : Selecciona categoría y fechas a programar
Admin -> UI : Clic en Generar Fixture Round Robin
UI -> Ctrl : generarFixtureRoundRobin(idCat)
activate Ctrl

Ctrl -> RepoEq : listarPorCategoria(idCat)
activate RepoEq
RepoEq --> Ctrl : listaEquiposHabilitados
deactivate RepoEq

Ctrl -> Ctrl : ejecutarAlgoritmoRoundRobin(listaEquipos)

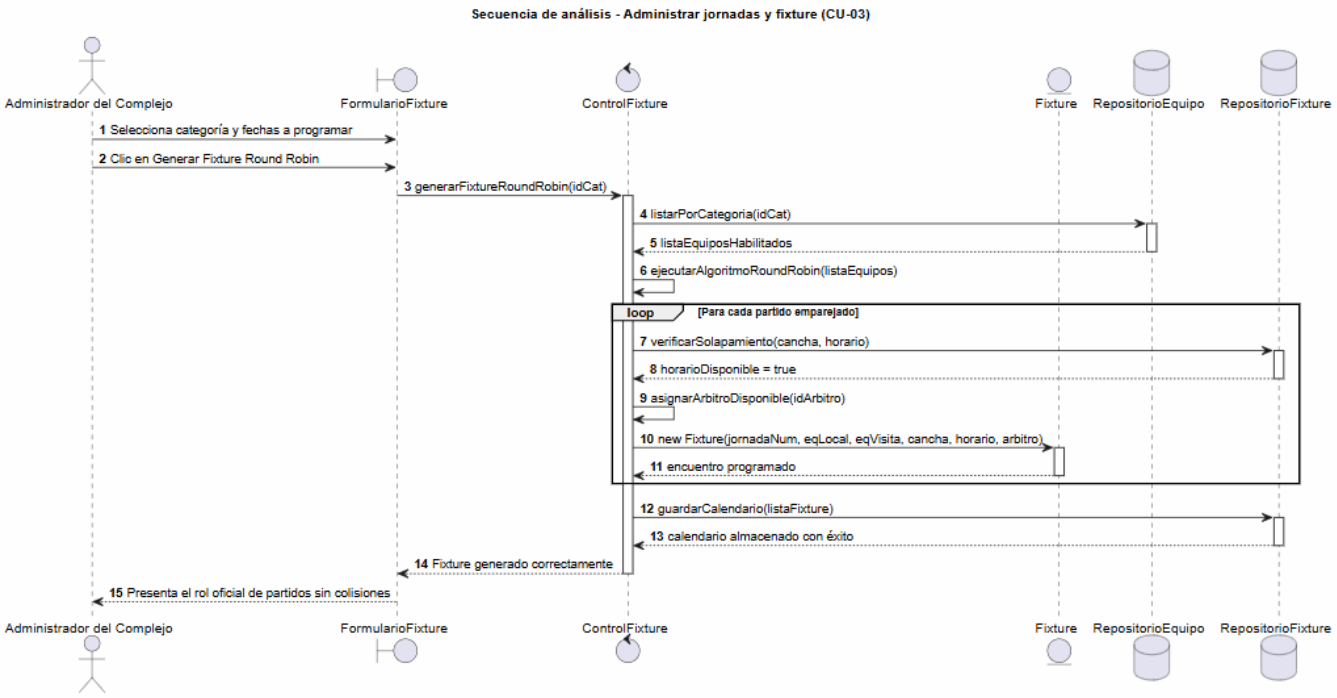
loop Para cada partido emparejado
    Ctrl -> RepoFix : verificarSolapamiento(cancha, horario)
    activate RepoFix
    RepoFix --> Ctrl : horarioDisponible = true
    deactivate RepoFix

    Ctrl -> Ctrl : asignarArbitroDisponible(idArbitro)
    Ctrl -> Fix : new Fixture(jornadaNum, eqLocal, eqVisita, cancha, horario, arbitro)
    activate Fix
    Fix --> Ctrl : encuentro programado
    deactivate Fix
end

Ctrl -> RepoFix : guardarCalendario(listaFixture)
activate RepoFix
RepoFix --> Ctrl : calendario almacenado con éxito
deactivate RepoFix

Ctrl --> UI : Fixture generado correctamente
deactivate Ctrl

UI --> Admin : Presenta el rol oficial de partidos sin colisiones
@enduml
```



8.4 Secuencia: Administrar tabla de posiciones (CU-04)

```
@startuml
title Secuencia de análisis - Administrar tabla de posiciones (CU-04)
autonumber
actor "Administrador del Complejo" as Admin
actor "Aficionado / Jugador" as Aficionado
boundary "FormularioTablaPosiciones" as UI
control "ControlTablaPosiciones" as Ctrl
entity "TablaPosiciones" as Tabl
database "RepositorioTablaPosiciones" as RepoTabl

Admin -> UI : Cierra formalmente las actas de juego digitales de la fecha
Admin -> UI : Clic en Recalcular y Auditar Posiciones
UI -> Ctrl : recalcularEstadisticas(idCat)
activate Ctrl

Ctrl -> RepoTabl : obtenerActasCerradas(idCat)
activate RepoTabl
RepoTabl --> Ctrl : listaActasPartidos
deactivate RepoTabl

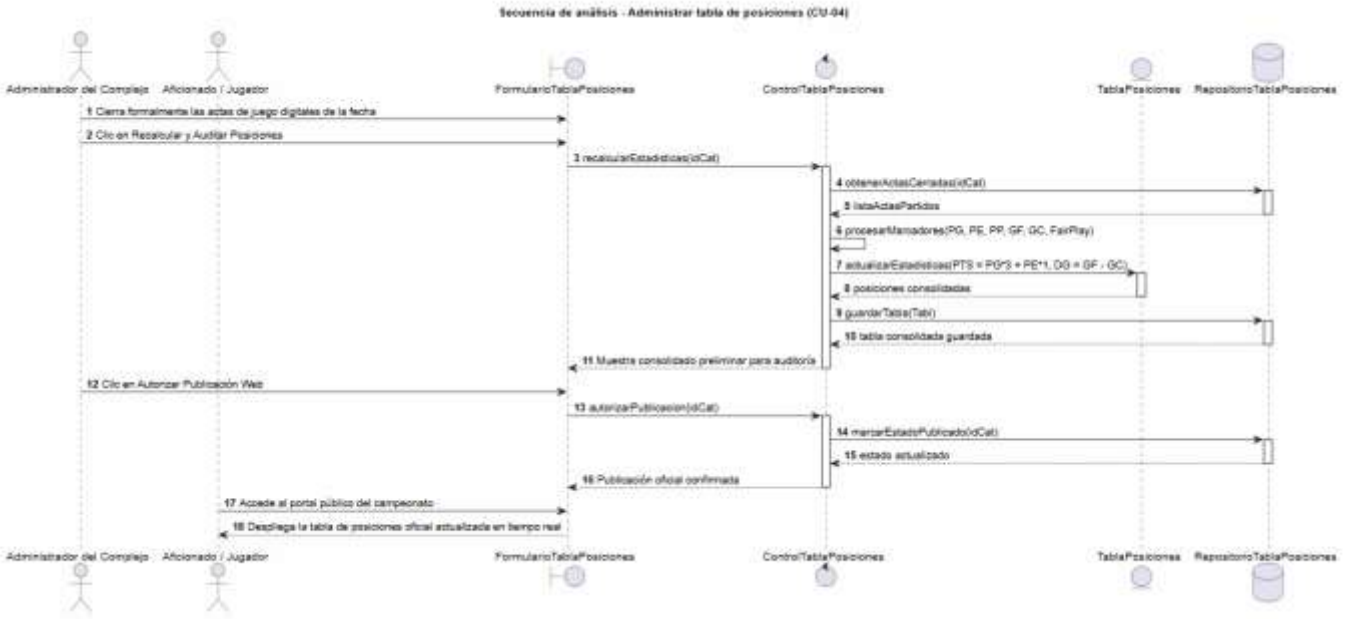
Ctrl -> Ctrl : procesarMarcadores(PG, PE, PP, GF, GC, FairPlay)
Ctrl -> Tabl : actualizarEstadisticas(PTS = PG*3 + PE*1, DG = GF - GC)
activate Tabl
Tabl --> Ctrl : posiciones consolidadas
deactivate Tabl
```

```
Ctrl -> RepoTabl : guardarTabla(Tabl)
activate RepoTabl
RepoTabl --> Ctrl : tabla consolidada guardada
deactivate RepoTabl

Ctrl --> UI : Muestra consolidado preliminar para auditoría
deactivate Ctrl

Admin -> UI : Clic en Autorizar Publicación Web
UI -> Ctrl : autorizarPublicacion(idCat)
activate Ctrl
Ctrl -> RepoTabl : marcarEstadoPublicado(idCat)
activate RepoTabl
RepoTabl --> Ctrl : estado actualizado
deactivate RepoTabl
Ctrl --> UI : Publicación oficial confirmada
deactivate Ctrl

Aficionado -> UI : Accede al portal público del campeonato
UI --> Aficionado : Despliega la tabla de posiciones oficial actualizada en tiempo real
@enduml
```



9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
CU-01: Administrar campeonato	El Administrador debe haber iniciado sesión con credenciales válidas en la plataforma.	Seleccionar Nuevo Campeonato -> ingresar nombre, categorías y cupos -> registrar fechas y costos ($60/$15) -> validar rango de fechas y costos -> verificar nombre único -> guardar en repositorio.	Campeonato creado formalmente y almacenado en estado "Inscripción Abierta".

Actualizar estudiante	El campeonato debe encontrarse en período de inscripción activa.	Ingresar datos del club -> cargar nómina de hasta 18 jugadores -> adjuntar cédula y foto (<= 2 MB) -> validar cédula (módulo 10) y peso -> auditar documentación -> generar firma QR HMAC -> guardar nómina.	Club deportivo acreditado y deportistas con carnets inteligentes QR emitidos.
CU-03: Administrar jornadas y fixture	Deben existir equipos inscritos y habilitados en la categoría seleccionada.	Seleccionar categoría -> disparar algoritmo Round Robin -> verificar disponibilidad de canchas -> asignar horarios libres -> designar árbitros sin solapamiento -> guardar fixture oficial.	Rol de partidos planificado de forma determinística, sin colisiones y publicado en el portal.
CU-04: Administrar tabla de posiciones	Se deben haber cerrado y firmado formalmente las actas digitales de la jornada deportiva.	Se deben haber cerrado y firmado formalmente las actas digitales de la jornada deportiva.	Se deben haber cerrado y firmado formalmente las actas digitales de la jornada deportiva.

10. Procedimiento recomendado para el estudiante

- Revisar la interfaz CASO e identificar campos de datos: ID, Nombre y Edad.
- Redactar requisitos funcionales de nivel 0 para Agregar, Actualizar, Eliminar y Mostrar Todo.
- Modelar los casos de uso en PlantUML y justificar únicamente las relaciones include necesarias.
- Derivar clases de análisis: una clase frontera para la interfaz, una clase control para coordinar la lógica, una entidad Estudiante y un repositorio o colección.
- Construir un diagrama de secuencia por cada caso de uso principal.
- Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases.
- Actualizar la matriz de trazabilidad para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- La entidad Estudiante conserva los atributos ID, Nombre y Edad.
- La validación de datos se aplica antes de agregar y actualizar.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actor, sistema, casos de uso y relaciones include justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.
Diagramas de secuencia	Modela flujos coherentes para Agregar, Actualizar, Eliminar y Mostrar Todo.	5 pts	Cuatro secuencias PlantUML.
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.
Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada operación CRUD principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

La integración sistemática entre casos de uso de análisis, clases de análisis (BCE + Repositorio) y diagramas de secuencia dinámica demuestra que el software no se concibe como artefactos aislados, sino como un modelo de ingeniería continuo y trazable:

- Los casos de uso de análisis definen qué valor funcional demanda cada actor externo (Administrador, Capitán, Aficionado).

- Las clases de análisis establecen qué objetos colaboran, asignando responsabilidades ortogonales entre la interfaz de usuario, los controladores de reglas, los datos del certamen y la capa de almacenamiento.

- Los diagramas de secuencia de análisis validan cómo interactúan cronológicamente dichos objetos, garantizando que los mensajes correspondan a métodos viables y que las validaciones de negocio se ejecuten antes de comprometer la persistencia.

Esta disciplina metodológica aplicada al **Sistema de Gestión de Campeonatos Deportivos (SGC-CMG)** previene errores tempranos de arquitectura, elimina el retrabajo en etapas avanzadas de codificación y asegura un paso transparente hacia el diseño de detalle y la implementación en el siguiente parcial.

15. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.